

DAY4

울산 4반 이름: 박인우

Chapter 1. 개인 수업 내용 이해 요약 정리

핵심은 Java/Spring Boot를 단순한 개발 언어·프레임워크가 아니라 기존 업무 데이터와 **AI** 기능을 연결하는 백엔드 중심 기술로 이해하는 것.

Spring Boot의 기본 계층은 Controller → Service → Repository → DB이다. Controller는 HTTP 요청·응답과 JSON 변환을 담당하고, Service는 회원가입·로그인·게시글 작성 같은 비즈니스 로직과 트랜잭션을 담당하며, Repository는 DB CRUD와 쿼리 실행을 담당한다. 의존 방향은 단방향이며 Controller가 Repository를 직접 호출하거나 Repository에 비즈니스 로직을 넣는 것은 피해야 한다. 코드를 읽을 때는 각 클래스의 **private final** 필드부터 확인하면 누구와 협업하는지 알 수 있고, 해당 타입의 파일로 이동해 같은 방식으로 따라가면 전체 관계도를 빠르게 파악할 수 있음.

데이터 객체도 역할에 따라 분리한다. @Entity는 DB 테이블과 매핑되는 영속 객체이고, DTO는 Request·Response처럼 계층 간 데이터를 전달하기 위한 객체다. 같은 데이터라도 회원가입, 로그인, 조회, 수정 등의 용도에 따라 DTO를 따로 만든다.

JPA는 Java Entity와 DB 테이블을 매핑해 SQL과 객체 변환 작업을 자동화한다.

JpaRepository<User, Long>을 상속하면 save, findById, findAll 같은 기본 CRUD가 자동 제공되고, findByUsername, existsByUsername처럼 규칙에 맞는 메서드 이름을 선언하면 Spring Data JPA가 필요한 쿼리도 자동 생성한다.

오늘 실습에서는 회원가입·로그인·게시글 작성 API를 만들고 app.http로 호출해 MariaDB에 실제 데이터가 저장되는 것까지 확인. 결국 앞으로 AI 기능도 이 구조를 버리는 것이 아니라, 기존 Service 흐름 안에 AI Service나 외부 API 호출을 추가해 업무 데이터와 **AI**를 연결하는 방식으로 확장하게 된다.

Chapter 2. 수업 내용 정리

우리는 현재 하고있는 업무에 ai기능을 추가해야된다. 현재 대부분은 java로 구성되어 있기 때문에 java를 알아줘야한다

ai로 업무관계자들에게 가치를 제공해야되는데 업무에 데이터를 가지고 하는 것에 있어서 중심이 자바가된다

개발자로 바라보지말고, ai 서비스의 백엔드로 바라봐야됨.(그래서 주로 아키텍처 얘기를 하는것)

그래서 나중에 이러한 요소와 기술이 나중에 어떤식으로 사용할 것인지를 고민해라

디렉토리 구조로 만듦 -> 확장 수정 용이

나눠놔야하니까 서로 쓸때 연결(@어노테이션 사용)

스프링부트가 @있는거 자동실행 되고 시작

세 가지 축(구조 / 흐름 / 확장)만 체화

1단계 : 관계도, 구조 파악

2단계 : 실제 동작 방식 : 이걸 어디서 예외가 날 수 있고, 어디서 정상 흐름인지" 추적할 수 있는 범용 독해력을 만드는 게 목적

3단계 : 절대 도메인 하나로 안 끝나고, 도메인마다 저장 방식 (JdbcTemplate vs JPA)이나 입력 방식 (폼 vs JSON)이 다를 수 있다.

"패턴은 반복되지만 세부 구현은 다를 수 있다"는 걸 미리 겪어봐서, 세 번째, 네 번째 도메인을 만나도 당황하지 않고 "뭐가 같고 뭐가 다른지"부터 빠르게 골라낼 수 있게 됨.

코드 흐름 읽는법

1. 클래스 맨 위, 생성자 바로 위에 있는 필드 목록부터 봅니다. (private final AuthService

authService; 같은 줄) à "이 클래스가 누구랑 협업하는지" 명단

2. 그 타입 이름을 파일 최상단 import 문에서 찾습니다. à 그 클래스가 몇 번째 폴더(패키지)

에 있는지 나옵니다

3. 그 경로로 직접 이동해서 파일을 엽니다. 그 파일도 또 자기만의 필드 목록이 있을 테니, 1번부터 반복합니다.

응답은 왜 항상 존재하는가?

응답을 직접 설계하지 않아도 **Spring Boot**(더 정확히는 **HTTP** 자체의 규칙)가 항상 "기본값"으로 뭔가 응답을 만들어서 보냅니다. 다만 그 기본값이 원하는 응답이 아닐 수 있다는 게 문제입니다.

HTTP는 요청을 보내면 반드시 상태 코드(200, 302, 404, 500 등)가 포함된 응답이 와야 하는 프로토콜입니다. "응답 안 함"이라는 선택지 자체가 없습니다. 그래서 개발자가 응답을 세세히 설계하지 않으면, **Spring Boot**가 상황에 맞는 기본 동작으로 대신 채워 넣습니다

Spring Boot가 자동으로 처리해주는 것과 무관하게, 아래 최소 기준에서 모두 "Yes"일 때 **REST**로 판단합니다.

- 1. 요청/응답이 구조화된 데이터(**JSON**)로 오가는가? (양식 데이터나 **HTML**이 아니라)
- 2. 응답이 데이터인가, 화면 이동 지시(**redirect**)인가?
- 3. 서버가 상태(세션)를 안 갖고 매 요청을 독립적으로 처리하는가?

직렬화(**Serialization**)란, 메모리 안에 있는 자바 객체를 네트워크로 전송 가능한 "**JSON 문자열**(텍스트)" 형태로 바꾸는 작업 (**HTTP**는 객체를 그대로 전송할 방법이 없고, 오직 텍스트(문자열)나 바이트만 주고받을 수 있음)

요청 : 완성된 **AuthCheckResponse** 객체를 그대로 반환하며 처리를 위임

응답 : **@RestController**가 이 객체를 **JSON** 문자열로 변환, 최종 **HTTP** 응답으로 전송

auth 도메인 (**JdbcUserRepository**) 구현체 직접 클래스 작성 필요, **JdbcTemplate** + 직접 **SQL** 작성, **SQL**을 세밀하게 제어 가능(대용량 데이터 만지기 용이,(인덱스사용..))

post 도메인 (PostRepository) : Spring Data JPA (SQL 자동 생성), 인터페이스만 있으면 Spring이 자동 구현, SQL을 세밀하게 제어 가능, 코드량이 훨씬 적고 빠르게 개발 가능

도메인 간 연결 : Authentication 재사용해서 해봄

계층형 아키텍처- 3계층 책임 분리

Presentation Layer @RestController HTTP 요청/응답, JSON 직렬화, 입력 검증, 비즈니스 로직 없음

Business Layer @Service 비즈니스 규칙, 트랜잭션, 여러 Repository 조율, HTTP/DB 개념 없음

Persistence Layer @Repository / JPA DB CRUD, 쿼리 실행, Entity 매핑, 비즈니스 로직 없음

단방향 의존 규칙: Controller → Service → Repository (역방향 절대 금지 / 계층 건너뛰기 금지)

Entity · DTO · VO - 계층 경계의 데이터 설계

왜 같은'데이터'인데 별도 객체로 존재하는가· 계층 경계를 넘을 때 변환(Mapping)이 필요한 이유

Entity @Entity : DB 테이블과 1:1 매핑, JPA가 관리하는 영속 객체

DTO : 계층 간 데이터 전달 전용, Request/Response 분리

VO : 불변(Immutable)(특정) 값 객체, 동등성은 값으로 판단

Entity를 Controller가 직접 반환하면? - DB 구조 노출 + 순환 참조 + 불필요한 필드 노출 → 반드시 DTO로 변환해서 반환

Repository - 데이터 접근을 인터페이스로 분리해서 사용

금지 패턴- 왜 위험한가

✗ Controller → Repository 직접 호출 → 비즈니스 로직이 Controller에 섞임, Service가 테스트 불가능

X Service → Controller 역방향 호출 → 순환 의존 발생, HTTP 개념이 Service에 침투

X Repository 안에 비즈니스 로직 → DB 변경 시 비즈니스 로직도 함께 수정

JPA 기초- Entity 매핑 · Spring Data JPA

ORM (Object-Relational Mapping) : Java 객체 ↔ DB 테이블 자동 매핑 : SQL 없이 Java 메서드로 DB 조작

@Entity : DB 테이블과 매핑되는 클래스 : @Id로 PK 지정, @Column으로 컬럼 매핑

영속성 컨텍스트 : JPA가 Entity를 관리하는 1차 캐시 : 변경 감지(dirty checking)로 자동 UPDATE

스프링 부트는 자동으로 resources, static, yml 등을 찾아 읽어서 동적으로 연결할 부분을 찾음

핵심 개념 : Spring Boot는 순수 Java에서 반복 구현하는 웹 서버 기능을 자동화한 프레임워크.

controller -> service -> repository(서비스와 db(DTO)간의 데이터 가공) -> db(서버에 저장)(Dao, Ds, DTO에 담음)

거기에 secrete 추가

JPA의 개요 : JPA는 Java 클래스의 객체·필드와 DB의 테이블·컬럼을 매핑하여, 개발자가 SQL과 객체 변환 코드를 직접 반복해서 작성하는 대신 Java 객체 중심으로 DB를 다룰 수 있게 해주는 기술

무조건 Entity는 DB 구조와 대응되게 만든다.

같은 데이터를 쓰더라도 용도마다 클래스를 각각 만들어서 써야됨!!!

DB와 연결된 Entity 객체에서 필요한 데이터를 꺼내 용도별 DTO 객체에 새로 담아서 사용

특히 JpaRepository<User, Long>를 상속하면 Spring Data JPA가 Repository 구현체와 기본 CRUD를 자동 제공하고, 추가로 findByUsername처럼 규칙에 맞는 메서드를 선언하면 그 이름을 분석해 필요한 조회 쿼리도 자동으로 처리해준다.

DB와 연결되는 Entity는 DB 구조에 맞춰 하나를 만들고, 같은 데이터를 사용하더라도 회원가입·로그인·조회·수정 등 용도마다 별도의 데이터 객체(DTO)를 먼저 정의한 뒤 각 메서드를 구현한다. Entity 자체를 모든 곳에서 직접 가져다 쓰거나 임의로 수정하지 않는다.

테이블 추가 → 대응 Entity 추가 → 해당 Entity를 다룰 Repository 추가

기능을 추가할 때 새로운 영속 데이터가 필요하면 그 데이터를 표현하는 Entity와 Repository부터 같이 만들어 활용하자

Chapter 3. 코드 이해(실습교수님)

build.gradle

application.yml에서 DB 접속 정보와 JPA 동작 방식을 설정하고, spring-boot-starter-data-jpa와 MariaDB JDBC Driver 의존성을 추가해서 Spring Boot가 Java Entity와 MariaDB를 연결할 준비를 한다.

Application.yml

DB 접속 정보, JPA/Hibernate 동작 방식, Spring 서버 포트를 설정

User.java

User Entity는 users100 테이블과 매핑되는 Java 객체이며, 각 필드와 JPA 어노테이션을 통해 PK, 중복 여부, NULL 허용 여부, 길이, 생성시간 등의 DB 구조와 저장 규칙을 정의

UserRepository.java

UserRepository는 User Entity를 대상으로 DB 작업을 수행하는 JPA Repository이며, JpaRepository<User, Long>를 상속하면 기본 CRUD가 자동 제공되고, findByUsername,

`existsByUsername`처럼 규칙에 맞는 메서드 이름을 선언하면 Spring Data JPA가 필요한 조회 로직까지 자동으로 만들어준다.

AuthService.java

어제는 `AuthService`가 `HashMap`에서 사용자를 직접 저장·조회했지만, 오늘은 `UserRepository`를 통해 JPA가 MariaDB의 `users100` 테이블을 저장·조회한다. `AuthService`는 회원가입·로그인이라는 업무 로직에 집중하고, 실제 DB 접근은 `Repository`에 맡긴다.

AuthController.java

`AuthController`는 `/api/auth/register`, `/api/auth/login` HTTP 요청을 받아 DTO로 변환하고, 실제 회원가입·로그인 처리는 `AuthService`에 위임한 뒤 결과만 HTTP 응답으로 반환하는 웹 계층

Post.java

새 기능이 새로운 데이터를 DB에 저장해야 하므로, 그 데이터를 표현하는 새 Entity인 `Post`를 먼저 만든다. 이후 `PostRepository`, `PostService`, `PostController`를 차례로 연결한다.

PostRepository.java

`PostRepository`는 `Post` Entity를 대상으로 DB 작업을 수행하는 JPA Repository이며, `JpaRepository<Post, Long>`을 상속함으로써 기본 CRUD 기능을 Spring Data JPA가 자동으로 제공

PostService.java

`PostService`는 `PostRequest`로 들어온 데이터를 `Post` Entity로 만들어 `Repository`에 저장하고, 저장된 Entity를 `PostResponse`로 변환해서 `Controller`에 반환하는 게시글 생성 비즈니스 로직을 담당

PostController.java

`PostController`는 `POST /api/posts` 요청을 받아 `JSON`을 `PostRequest DTO`로 변환하고, 실제 게시글 생성은 `PostService`에 위임한 뒤 생성된 `PostResponse`를 `201 Created` 상태와 함께 반환하는 웹 계층

최종정리

Controller → Service → Repository → Entity → JPA → DB

`application.yml`로 MariaDB/JPA 연결을 설정하고, User/Post별로 `Entity → Repository → Service → Controller` 구조를 만든 뒤 `app.http`에서 회원가입·로그인·게시글 작성 API를 호출하여 JPA가 실제 SQL을 생성하고 `users100`, `posts100` 테이블에 데이터가 저장되는 것까지 확인

Chapter 4. 궁금한점 및 보충

rest api와 전통적인 방식? 의 차이점

전통 MVC = 서버가 데이터 처리 + 화면 흐름까지 담당

REST API = 서버는 데이터와 처리 결과를 제공하고, 화면 흐름은 프론트엔드가 담당

전통적인 MVC	REST API
-----	-----
서버가 화면 흐름까지 많이 담당	서버는 주로 데이터/API 담당
HTML/Form 중심	JSON 중심
`redirect`, View 이름 반환	객체/DTO 반환
서버와 화면이 강하게 연결됨	프론트와 백엔드가 분리됨
`sendRedirect("/login")`	`200`, `201` + JSON

그럼 REST API에서는 DTO를 주로 이용해서 동작하나?

REST API에서는 DTO를 굉장히 자주 사용. 특히 Web 계층의 입구와 출구에서 많이 씀

REST API에서는 DTO가 요청 JSON을 받아들이고, 처리 결과를 응답 JSON으로 내보내는 Web 계층의 데이터 그릇 역할을 주로 한다

도메인?

Domain = 프로그램이 실제로 해결하려는 업무 세계의 핵심 대상과 그 대상이 가지는 규칙을 코드로 표현한 것.

각 패키지(포스트, auth)의 @Service, @Repository, @Controller 등의 객체를 Spring Bean으로 만들고, 필요한 객체를 생성자 DI로 연결

순환의존?

순환 의존 = 서로가 서로를 필요로 하는 구조.

$A \rightarrow B \rightarrow A$ 같은 연결은 피하고, 가능하면 $A \rightarrow B \rightarrow C$ 처럼 한 방향으로 흐르게 설계한다.

그리들 의존성 주입?

Gradle dependency = 프로젝트에서 사용할 외부 기능/라이브러리를 선언하는 것.

Gradle이 그 라이브러리를 자동으로 받아서 빌드와 실행에 연결해준다.

SpringBoot 실행 → 설정 읽기(yml, ...) → Bean 스캔/생성 → DI → Tomcat 실행 → HTTP 요청 → Security → DispatcherServlet → Controller → Service → Repository → DB → HTTP 응답

